

# NAME

samtools – Utilities for the Sequence Alignment/Map (SAM) format

## SYNOPSIS

samtools addreplacerg (samtools-addreplacerg.html) -r 'ID:fish' -r 'LB:1334' -r 'SM:alpha' -o output.bam  
input.bam

samtools ampliconclip (samtools-ampliconclip.html) -b bed.file input.bam

samtools ampliconstats (samtools-ampliconstats.html) primers.bed in.bam

samtools bedcov (samtools-bedcov.html) aln.sorted.bam

samtools calmd (samtools-calmd.html) in.sorted.bam ref.fasta

samtools cat (samtools-cat.html) out.bam in1.bam in2.bam in3.bam

samtools checksum (samtools-checksum.html) in.bam

samtools collate (samtools-collate.html) -o aln.name\_collated.bam aln.sorted.bam

samtools consensus (samtools-consensus.html) -o out.fasta in.bam

samtools coverage (samtools-coverage.html) aln.sorted.bam

samtools cram-size (samtools-cram-size.html) -v -o out.size in.cram

samtools depad (samtools-depad.html) input.bam

samtools depth (samtools-depth.html) aln.sorted.bam

samtools dict (samtools-dict.html) -a GRCh38 -s "Homo sapiens" ref.fasta

samtools faidx (samtools-faidx.html) ref.fasta

samtools fasta (samtools-fasta.html) input.bam > output.fasta

samtools fastq (samtools-fastq.html) input.bam > output.fastq

samtools fixmate (samtools-fixmate.html) in.namesorted.sam out.bam

samtools flags (samtools-flags.html) PAIRED,UNMAP,MUNMAP

samtools flagstat (samtools-flagstat.html) aln.sorted.bam

samtools fqidx (samtools-fqidx.html) ref.fastq

samtools head (samtools-head.html) in.bam

samtools idxstats (samtools-idxstats.html) aln.sorted.bam

samtools import (samtools-import.html) input.fastq > output.bam

samtools index (samtools-index.html) aln.sorted.bam

samtools markdup (samtools-markdup.html) in.alnsorted.bam out.bam

samtools merge (samtools-merge.html) out.bam in1.bam in2.bam in3.bam

samtools mpileup (samtools-mpileup.html) -f ref.fasta -r chr3:1,000-2,000 in1.bam in2.bam

 samtools phase (samtools-phase.html) input.bam  
samtools quickcheck (samtools-quickcheck.html) in1.bam in2.cram  
samtools reference (samtools-reference.html) -o ref.fa in.cram  
samtools reheader (samtools-reheader.html) in.header.sam in.bam > out.bam  
samtools reset (samtools-reset.html) -o /tmp/reset.bam processed.bam  
samtools samples (samtools-samples.html) input.bam  
samtools sort (samtools-sort.html) -T /tmp/aln.sorted -o aln.sorted.bam aln.bam  
samtools split (samtools-split.html) merged.bam  
samtools stats (samtools-stats.html) aln.sorted.bam  
samtools targetcut (samtools-targetcut.html) input.bam  
samtools tview (samtools-tview.html) aln.sorted.bam ref.fasta  
samtools view (samtools-view.html) -bt ref\_list.txt -o aln.bam aln.sam.gz

## DESCRIPTION

Samtools is a set of utilities that manipulate alignments in the SAM (Sequence Alignment/Map), BAM, and CRAM formats. It converts between the formats, does sorting, merging and indexing, and can retrieve reads in any regions swiftly.

Samtools is designed to work on a stream. It regards an input file '-' as the standard input (stdin) and an output file '-' as the standard output (stdout). Several commands can thus be combined with Unix pipes. Samtools always output warning and error messages to the standard error output (stderr).

Samtools is also able to open files on remote FTP or HTTP(S) servers if the file name starts with 'ftp://', 'http://', etc. Samtools checks the current working directory for the index file and will download the index upon absence. Samtools does not retrieve the entire alignment file unless it is asked to do so.

If an index is needed, samtools looks for the index suffix appended to the filename, and if that isn't found it tries again without the filename suffix (for example **in.bam.bai** followed by **in.bai**). However if an index is in a completely different location or has a different name, both the main data filename and index filename can be pasted together with **##idx##**. For example **/data/in.bam##idx##/indices/in.bam.bai** may be used to explicitly indicate where the data and index files reside.

## COMMANDS

Each command has its own man page which can be viewed using e.g. **man samtools-view** or with a recent GNU man using **man samtools view**. Below we have a brief summary of syntax and sub-command description.

Options common to all sub-commands are documented below in the GLOBAL COMMAND OPTIONS section.

**view**                    samtools view [*options*] *in.sam|in.bam|in.cram* [*region...*]

With no options or regions specified, prints all alignments in the specified input alignment file (in SAM, BAM, or CRAM format) to standard output in SAM format (with no header by default).



You may specify one or more space-separated region specifications after the input filename to restrict output to only those alignments which overlap the specified region(s). Use of region specifications requires a coordinate-sorted and indexed input file.

Options exist to change the output format from SAM to BAM or CRAM, so this command also acts as a file format conversion utility.

#### tvview

`samtools tvview [-p chr:pos] [-s STR] [-d display] <in.sorted.bam> [ref.fasta]`

Text alignment viewer (based on the ncurses library). In the viewer, press '?' for help and press 'g' to check the alignment start from a region in the format like 'chr10:10,000,000' or '=10,000,000' when viewing the same reference sequence.

#### quickcheck

`samtools quickcheck [options] in.sam|in.bam|in.cram [ ... ]`

Quickly check that input files appear to be intact. Checks that beginning of the file contains a valid header (all formats) containing at least one target sequence and then seeks to the end of the file and checks that an end-of-file (EOF) is present and intact (BAM only).

Data in the middle of the file is not read since that would be much more time consuming, so please note that this command will not detect internal corruption, but is useful for testing that files are not truncated before performing more intensive tasks on them.

This command will exit with a non-zero exit code if any input files don't have a valid header or are missing an EOF block. Otherwise it will exit successfully (with a zero exit code).

#### checksum

`samtools checksum [options] in.sam|in.bam|in.cram`

`samtools checksum` produces a CRC32 based checksum of data contained within a BAM file. This can either be order and orientation agnostic for purposes of validating all the sequencing data has passed through the entire pipeline from FASTQ through alignment and sorting, or full alignment information and order aware for the purposes of validating format conversions and while file data processing.

#### head

`samtools head [options] in.sam|in.bam|in.cram`

Prints the input file's headers and optionally also its first few alignment records. This command always displays the headers as they are in the file, never adding an extra @PG header itself.

#### index

`samtools index [-bc] [-m INT] aln.sam.gz|aln.bam|aln.cram [out.index]`

Index a coordinate-sorted SAM, BAM or CRAM file for fast random access. Note for SAM this only works if the file has been BGZF compressed first. (Starting from Samtools 1.16, this command can also be given several alignment filenames, which are indexed individually.)

This index is needed when *region* arguments are used to limit **samtools view** and similar commands to particular regions of interest.

If an output filename is given, the index file will be written to *out.index*. Otherwise, for a CRAM file *aln.cram*, index file *aln.cram.crai* will be created; for a BAM or SAM file *aln.bam*, either *aln.bam.bai* or *aln.bam.csi* will be created, depending on the index format selected.

#### sort

`samtools sort [-I level] [-m maxMem] [-o out.bam] [-O format] [-n] [-t tag] [-T tmpprefix] [-@ threads] [in.sam|in.bam|in.cram]`

Sort alignments by leftmost coordinates, or by read name when **-n** is used. An appropriate @HD-SO sort order header tag will be added or an existing one updated if necessary.



The sorted output is written to standard output by default, or to the specified file (*out.bam*) when **-o** is used. This command will also create temporary files *tmpprefix.%d.bam* as needed when the entire alignment data cannot fit into memory (as controlled via the **-m** option).

Consider using **samtools collate** instead if you need name collated data without a full lexicographical sort.

Note that if the sorted output file is to be indexed with **samtools index**, the default coordinate sort must be used. Thus the **-n** and **-t** options are incompatible with **samtools index**.

#### **collate**

`samtools collate [options] in.sam|in.bam|in.cram [<prefix>]`

Shuffles and groups reads together by their names. A faster alternative to a full query name sort, **collate** ensures that reads of the same name are grouped together in contiguous groups, but doesn't make any guarantees about the order of read names between groups.

The output from this command should be suitable for any operation that requires all reads from the same template to be grouped together.

#### **cram-size**

`samtools cram-size [options] in.cram`

Produces a summary of CRAM block Content ID numbers and their associated Data Series stored within them. Optionally a more detailed breakdown of how each data series is encoded per container may also be listed using the **-e** or **--encodings** option.

#### **idxstats**

`samtools idxstats in.sam|in.bam|in.cram`

Retrieve and print stats in the index file corresponding to the input file. Before calling `idxstats`, the input BAM file should be indexed by `samtools index`.

If run on a SAM or CRAM file or an unindexed BAM file, this command will still produce the same summary statistics, but does so by reading through the entire file. This is far slower than using the BAM indices.

The output is TAB-delimited with each line consisting of reference sequence name, sequence length, # mapped reads and # unmapped reads. It is written to stdout.

#### **flagstat**

`samtools flagstat in.sam|in.bam|in.cram`

Does a full pass through the input file to calculate and print statistics to stdout.

Provides counts for each of 13 categories based primarily on bit flags in the FLAG field. Each category in the output is broken down into QC pass and QC fail, which is presented as "#PASS + #FAIL" followed by a description of the category.

#### **flags**

`samtools flags INT|STR[,...]`

Convert between textual and numeric flag representation.

#### **FLAGS:**

|             |             |                                                        |
|-------------|-------------|--------------------------------------------------------|
| <b>0x1</b>  | PAIRED      | paired-end (or multiple-segment) sequencing technology |
| <b>0x2</b>  | PROPER_PAIR | each segment properly aligned according to the aligner |
| <b>0x4</b>  | UNMAP       | segment unmapped                                       |
| <b>0x8</b>  | MUNMAP      | next segment in the template unmapped                  |
| <b>0x10</b> | REVERSE     | SEQ is reverse complemented                            |



|              |               |                                                                 |
|--------------|---------------|-----------------------------------------------------------------|
| <b>0x20</b>  | MREVERSE      | SEQ of the next segment in the template is reverse complemented |
| <b>0x40</b>  | READ1         | the first segment in the template                               |
| <b>0x80</b>  | READ2         | the last segment in the template                                |
| <b>0x100</b> | SECONDARY     | secondary alignment                                             |
| <b>0x200</b> | QCFAIL        | not passing quality controls                                    |
| <b>0x400</b> | DUP           | PCR or optical duplicate                                        |
| <b>0x800</b> | SUPPLEMENTARY | supplementary alignment                                         |

**stats** `samtools stats [options] in.sam|in.bam|in.cram [region...]`

`samtools stats` collects statistics from BAM files and outputs in a text format. The output can be visualized graphically using `plot-bamstats`.

**bedcov** `samtools bedcov [options] region.bed in1.sam|in1.bam|in1.cram[...]`

Reports the total read base count (i.e. the sum of per base read depths) for each genomic region specified in the supplied BED file. The regions are output as they appear in the BED file and are 0-based. Counts for each alignment file supplied are reported in separate columns.

**depth** `samtools depth [options] [in1.sam|in1.bam|in1.cram [in2.sam|in2.bam|in2.cram] [...]]`

Computes the read depth at each position or region.

**ampliconstats** `samtools ampliconstats [options] primers.bed in.sam|in.bam|in.cram[...]`

`samtools ampliconstats` collects statistics from one or more input alignment files and produces tables in text format. The output can be visualized graphically using `plot-ampliconstats`.

The alignment files should have previously been clipped of primer sequence, for example by **samtools ampliconclip** and the sites of these primers should be specified as a bed file in the arguments.

**mpileup** `samtools mpileup [-EB] [-C capQcoef] [-r reg] [-f in.fa] [-l list] [-Q minBaseQ] [-q minMapQ] in.bam [in2.bam [...]]`

Generate textual pileup for one or multiple BAM files. For VCF and BCF output, please use the **bcftools mpileup** command instead. Alignment records are grouped by sample (SM) identifiers in @RG header lines. If sample identifiers are absent, each input file is regarded as one sample.

See the `samtools-mpileup` man page for a description of the pileup format and options.

**consensus** `samtools consensus [options] in.bam`

Generate consensus from a SAM, BAM or CRAM file based on the contents of the alignment records. The consensus is written either as FASTA, FASTQ, or a pileup oriented format.

The default output for FASTA and FASTQ formats include one base per non-gap consensus. Hence insertions with respect to the aligned reference will be included and deletions removed. This behaviour can be adjusted.

Two consensus calling algorithms are offered. The default computes a heterozygous consensus in a Bayesian manner, derived from the "Gap5" consensus algorithm. A simpler base frequency counting method is also available.

**reference** `samtools reference [options] in.bam`



Generate a reference from a SAM, BAM or CRAM file based on the contents of the SEQUENCE field and the MD:Z: auxiliary tags, or from the embedded reference blocks within a CRAM file (provided it was constructed using the **embed\_ref=1** option).

**coverage** `samtools coverage [options] [in1.sam|in1.bam|in1.cram [in2.sam|in2.bam|in2.cram] [...]]`

Produces a histogram or table of coverage per chromosome.

**merge** `samtools merge [-nur1f] [-h inh.sam] [-t tag] [-R reg] [-b list] out.bam in1.bam [in2.bam in3.bam ... inN.bam]`

Merge multiple sorted alignment files, producing a single sorted output file that contains all the input records and maintains the existing sort order.

If **-h** is specified the @SQ headers of input files will be merged into the specified header, otherwise they will be merged into a composite header created from the input headers. If the @SQ headers differ in order this may require the output file to be re-sorted after merge.

The ordering of the records in the input files must match the usage of the **-n** and **-t** command-line options. If they do not, the output order will be undefined. See **sort** for information about record ordering.

**split** `samtools split [options] merged.sam|merged.bam|merged.cram`

Splits a file by read group, producing one or more output files matching a common prefix (by default based on the input filename) each containing one read-group.

**cat** `samtools cat [-b list] [-h header.sam] [-o out.bam] in1.bam in2.bam [ ... ]`

Concatenate BAMs or CRAMs. Although this works on either BAM or CRAM, all input files must be the same format as each other. The sequence dictionary of each input file must be identical, although this command does not check this. This command uses a similar trick to **reheader** which enables fast BAM concatenation.

**import** `samtools import [options] in.fastq [ ... ]`

Converts one or more FASTQ files to unaligned SAM, BAM or CRAM. These formats offer a richer capability of tracking sample meta-data via the SAM header and per-read meta-data via the auxiliary tags. The **fastq** command may be used to reverse this conversion.

**fastq/a** `samtools fastq [options] in.bam`  
`samtools fasta [options] in.bam`

Converts a BAM or CRAM into either FASTQ or FASTA format depending on the command invoked. The files will be automatically compressed if the file names have a .gz, .bgz, or .bgzf extension.

The input to this program must be collated by name. Use **samtools collate** or **samtools sort -n** to ensure this.

**faidx** `samtools faidx <ref.fasta> [region1 [...]]`

Index reference sequence in the FASTA format or extract subsequence from indexed reference sequence. If no region is specified, **faidx** will index the file and create `<ref.fasta>.fai` on the disk. If regions are specified, the subsequences will be retrieved and printed to stdout in the FASTA format.

The input file can be compressed in the **BGZF** format.

FASTQ files can be read and indexed by this command. Without using **--fastq** any extracted subsequence will be in FASTA format.



## fqidx

`samtools fqidx <ref.fastq> [region1 [...]]`

Index reference sequence in the FASTQ format or extract subsequence from indexed reference sequence. If no region is specified, **fqidx** will index the file and create `<ref.fastq>.fai` on the disk. If regions are specified, the subsequences will be retrieved and printed to stdout in the FASTQ format.

The input file can be compressed in the **BGZF** format.

**samtools fqidx** should only be used on fastq files with a small number of entries. Trying to use it on a file containing millions of short sequencing reads will produce an index that is almost as big as the original file, and searches using the index will be very slow and use a lot of memory.

## dict

`samtools dict ref.fasta|ref.fasta.gz`

Create a sequence dictionary file from a fasta file.

## calmd

`samtools calmd [-Eeubr] [-C capQcoef] aln.bam ref.fasta`

Generate the MD tag. If the MD tag is already present, this command will give a warning if the MD tag generated is different from the existing tag. Output SAM by default.

Calmd can also read and write CRAM files although in most cases it is pointless as CRAM recalculates MD and NM tags on the fly. The one exception to this case is where both input and output CRAM files have been / are being created with the *no\_ref* option.

## fixmate

`samtools fixmate [-rpcm] [-O format] in.nameSrt.bam out.bam`

Fill in mate coordinates, ISIZE and mate related flags from a name-sorted alignment.

## markdup

`samtools markdup [-l length] [-r] [-s] [-T] [-S] in.algsort.bam out.bam`

Mark duplicate alignments from a coordinate sorted file that has been run through **samtools fixmate** with the **-m** option. This program relies on the MC and ms tags that fixmate provides.

## rmdup

`samtools rmdup [-sS] <input.srt.bam> <out.bam>`

**This command is obsolete. Use markdup instead.**

## addreplacerg

`samtools addreplacerg [-r rg-line | -R rg-ID] [-m mode] [-l level] [-o out.bam] in.bam`

Adds or replaces read group tags in a file.

## reheader

`samtools reheader [-iP] in.header.sam in.bam`

Replace the header in *in.bam* with the header in *in.header.sam*. This command is much faster than replacing the header with a BAM→SAM→BAM conversion.

By default this command outputs the BAM or CRAM file to standard output (stdout), but for CRAM format files it has the option to perform an in-place edit, both reading and writing to the same file. No validity checking is performed on the header, nor that it is suitable to use with the sequence data itself.

## targetcut

`samtools targetcut [-Q minBaseQ] [-i inPenalty] [-0 em0] [-1 em1] [-2 em2] [-f ref] in.bam`

This command identifies target regions by examining the continuity of read depth, computes haploid consensus sequences of targets and outputs a SAM with each sequence corresponding to a target. When option **-f** is in use, BAQ will be applied. This command is **only** designed for cutting fosmid clones from fosmid pool sequencing [Ref. Kitzman et al. (2010)].





## phase

samtools phase [-AF] [-k *len*] [-b *prefix*] [-q *minLOD*] [-Q *minBaseQ*] *in.bam*

Call and phase heterozygous SNPs.

## depad

samtools depad [-SsCu1] [-T *ref.fa*] [-o *output*] *in.bam*

Converts a BAM aligned against a padded reference to a BAM aligned against the depadded reference. The padded reference may contain verbatim "\*" bases in it, but "\*" bases are also counted in the reference numbering. This means that a sequence base-call aligned against a reference "\*" is considered to be a cigar match ("M" or "X") operator (if the base-call is "A", "C", "G" or "T"). After depadding the reference "\*" bases are deleted and such aligned sequence base-calls become insertions. Similarly transformations apply for deletions and padding cigar operations.

## ampliconclip

samtools ampliconclip [-o *out.file*] [-f *stat.file*] [--soft-clip] [--hard-clip] [--both-ends] [--strand] [--clipped] [--fail] [--no-PG] -b *bed.file in.file*

Clip reads in a SAM compatible file based on data from a BED file.

## samples

samtools samples [-o *out.file*] [-i] [-T *TAG*] [-f *refs.fasta*] [-F *refs\_list*] [-X]

Prints the samples from alignment files

## reset

samtools reset [-o *FILE*] [-x/--remove-tag *tag\_list*] [--keep-tag *tag\_list*] [--reject-PG *pgid*] [--no-RG] [--no-PG] [...]

Removes alignment information from records, producing an unaligned SAM, BAM or CRAM file. Flags are reset, header tags are updated or removed as appropriate, and auxiliary tags are removed or retained as specified. Note that the sort order is unchanged.

# SAMTOOLS OPTIONS

These are options that are passed after the **samtools** command, before any sub-command is specified.

### help, --help

Display a brief usage message listing the samtools commands available. If the name of a command is also given, e.g., **samtools help view**, the detailed usage message for that particular command is displayed.

### --version

Display the version numbers and copyright information for samtools and the important libraries used by samtools.

### --version-only

Display the full samtools version number in a machine-readable format.

# GLOBAL COMMAND OPTIONS

Several long-options are shared between multiple samtools sub-commands: **--input-fmt**, **--input-fmt-option**, **--output-fmt**, **--output-fmt-option**, **--reference**, **--write-index**, and **--verbosity**. The input format is auto-detected and specifying the format is unnecessary, so this option is rarely offered. Note that not all subcommands have all options. Consult the subcommand help for more details.

Format strings recognised are "sam", "sam.gz", "bam" and "cram". They may be followed by a comma separated list of options as *key* or *key=value*. See below for examples.

The **fmt-option** arguments accept either a single *option* or *option=value*. Note that some options only work on some file formats and only on read or write streams. If value is unspecified for a boolean option, the value is assumed to be 1. The valid options are as follows.

**level=INT**





Output only. Specifies the compression level from 1 to 9, or 0 for uncompressed. If the output format is SAM, this also enables BGZF compression, otherwise SAM defaults to uncompressed.

**nthreads=INT** Specifies the number of threads to use during encoding and/or decoding. For BAM this will be encoding only. In CRAM the threads are dynamically shared between encoder and decoder.

**filter=STRING** Apply filter STRING to all incoming records, rejecting any that do not satisfy the expression. See the FILTER EXPRESSIONS section below for specifics.

**reference=fasta\_file**  
Specifies a FASTA reference file for use in CRAM encoding or decoding. It usually is not required for decoding except in the situation of the MD5 not being obtainable via the REF\_PATH or REF\_CACHE environment variables.

**decode\_md=0|1** CRAM input only; defaults to 1 (on). CRAM does not typically store MD and NM tags, preferring to generate them on the fly. When this option is 0 missing MD, NM tags will not be generated. It can be particularly useful when combined with a file encoded using store\_md=1 and store\_nm=1.

**store\_md=0|1** CRAM output only; defaults to 0 (off). CRAM normally only stores MD tags when the reference is unknown and lets the decoder generate these values on-the-fly (see decode\_md).

**store\_nm=0|1** CRAM output only; defaults to 0 (off). CRAM normally only stores NM tags when the reference is unknown and lets the decoder generate these values on-the-fly (see decode\_md).

**ignore\_md5=0|1** CRAM input only; defaults to 0 (off). When enabled, md5 checksum errors on the reference sequence and block checksum errors within CRAM are ignored. Use of this option is strongly discouraged.

**required\_fields=bit-field**  
CRAM input only; specifies which SAM columns need to be populated. By default all fields are used. Limiting the decode to specific columns can have significant performance gains. The bit-field is a numerical value constructed from the following table.

|               |           |
|---------------|-----------|
| <b>0x1</b>    | SAM_QNAME |
| <b>0x2</b>    | SAM_FLAG  |
| <b>0x4</b>    | SAM_RNAME |
| <b>0x8</b>    | SAM_POS   |
| <b>0x10</b>   | SAM_MAPQ  |
| <b>0x20</b>   | SAM_CIGAR |
| <b>0x40</b>   | SAM_RNEXT |
| <b>0x80</b>   | SAM_PNEXT |
| <b>0x100</b>  | SAM_TLEN  |
| <b>0x200</b>  | SAM_SEQ   |
| <b>0x400</b>  | SAM_QUAL  |
| <b>0x800</b>  | SAM_AUX   |
| <b>0x1000</b> | SAM_RGAUX |

**name\_prefix=string**  
CRAM input only; defaults to output filename. Any sequences with auto-generated read names will use *string* as the name prefix.

**multi\_seq\_per\_slice=0|1**



CRAM output only; defaults to 0 (off). By default CRAM generates one container per reference sequence, except in the case of many small references (such as a fragmented assembly).

**version=***major.minor*

CRAM output only. Specifies the CRAM version number. Acceptable values are "2.1", "3.0", and "3.1".

**seqs\_per\_slice=***INT*

CRAM output only; defaults to 10000.

**slices\_per\_container=***INT*

CRAM output only; defaults to 1. The effect of having multiple slices per container is to share the compression header block between multiple slices. This is unlikely to have any significant impact unless the number of sequences per slice is reduced. (Together these two options control the granularity of random access.)

**embed\_ref=***0|1* CRAM output only; defaults to 0 (off). If 1, this will store portions of the reference sequence in each slice, permitting decode without having requiring an external copy of the reference sequence.

**no\_ref=***0|1* CRAM output only; defaults to 0 (off). If 1, sequences will be stored verbatim with no reference encoding. This can be useful if no reference is available for the file.

**use\_bzip2=***0|1* CRAM output only; defaults to 0 (off). Permits use of bzip2 in CRAM block compression.

**use\_lzma=***0|1* CRAM output only; defaults to 0 (off). Permits use of lzma in CRAM block compression.

**use\_arith=***0|1* CRAM  $\geq$  3.1 output only; enables use of arithmetic entropy coding in CRAM block compression. This is off by default, but enabled for archive mode. This is significantly slower but sometimes smaller than the standard rANS entropy encoder.

**use\_fqz=***0|1* CRAM  $\geq$  3.1 output only; enables and disables the fqzcomp quality compression method. This is on by default for version 3.1 and above only when the small and archive profiles are in use.

**use\_tok=***0|1* CRAM  $\geq$  3.1 output only; enables and disables the name tokeniser compression method. This is on by default for version 3.1 and above.

**lossy\_names=***0|1*

CRAM output only; defaults to 0 (off). If 1, templates with all members within the same CRAM slice will have their read names removed. New names will be automatically generated during decoding. Also see the **name\_prefix** option.

**fast, normal, small, archive**

CRAM output only. Set the CRAM compression profile. This is a simplified way of setting many output options at once. It changes the following options according to the profile in use. The "normal" profile is the default.

| Option                | fast  | normal | small | archive       |
|-----------------------|-------|--------|-------|---------------|
| <b>level</b>          | 1     | 5      | 6     | 7             |
| <b>use_bzip2</b>      | off   | off    | on    | on            |
| <b>use_lzma</b>       | off   | off    | off   | on if level>7 |
| <b>use_tok(*)</b>     | off   | on     | on    | on            |
| <b>use_fqz(*)</b>     | off   | off    | on    | on            |
| <b>use_arith(*)</b>   | off   | off    | off   | on            |
| <b>seqs_per_slice</b> | 10000 | 10000  | 25000 | 100000        |

(\*) **use\_tok**, **use\_fqz** and **use\_arith** are only enabled for CRAM version 3.1 and above.



The **level** listed is only the default value, and will not be set if it has been explicitly changed already. Additionally **bases\_per\_slice** is set to **500\*seqs\_per\_slice** unless previously explicitly set.

- fastq\_name2** FASTQ input only. Indicates that the names are not the first word in the header, but the second. This is a FASTQ variant commonly used in the SRA and ENA archives.
- fastq\_casava** FASTQ input and output only. The Illumina CASAVA identifiers are stored in the second word of the FASTQ header lines and store read meta-data. The CASAVA tag defines the data held in the READ1, READ2 and QCFAIL flags and the barcode auxiliary tag ("BC" by default). This option may be used to both read and write CASAVA identifiers.

**fastq\_barcode=TAG**

FASTQ input and output only. When the **fastq\_casava** option is used, this controls the name of the barcode aux tag to be used. *TAG* defaults to "BC" if not specified.

- fastq\_aux=LIST** FASTQ input and output only. Processes SAM format auxiliary tags following the other fields on the record identifier lines. If no *=LIST* is specified or *LIST* is "1" then all aux tags listed are copied to/from the SAM record. Otherwise it is a comma separated list of 2-letter tag types and is used to control which tags are processed with any others being omitted.

Note as commas are used to separate options in the **--output-fmt** string detailing file format and options combined together, you will need to use the **--output-fmt-option** option if you want to specify a comma separated list of tag types.

- fastq\_rnum** FASTQ output only. If set, paired reads will have "/1" and "/2" appended to their read names. This has no effect on unpaired reads. When reading FASTQ these suffixes are automatically detected and processed irrespective of the **fastq\_rnum** option.

**fastq\_umi=TAGLIST**

FASTQ input and output only. When reading from a FASTQ file this indicates to extract the UMI tag from the read name and to put it in the TAG specified (which defaults to 'RX' if no tag name is given). The UMI is assumed to be the 8th colon-separated element, conforming to Illumina BCL to FASTQ conversion specifications. However see the **fastq\_umi\_regexp** option for altering this.

When converting from SAM to FASTQ, *TAGLIST* is a comma separated list of tags which are checked in turn for their presence. The string from the first tag found is then appended to the end of the read-name. There is no regexp available in this case and the data is always appended to the end of the name, or if hash-number is present (for example 'name#49') to just prior to the hash character. The *TAGLIST* defaults to 'OX,RX'.

**fastq\_umi\_regex=REGEX**

FASTQ input only. Specifies the regular expression used for finding UMI strings in a read name. Any text within the single bracketted element will be used as the UMI string. Text matched by that string will be removed from the read name, with anything to the right of it being moved leftwards. This defaults to `^[^:]+:[^:]+:[^:]+:[^:]+:[^:]+:[^:]+:[^:]+:[^:]+([[:#]]+)`.

For example:

```
samtools view --input-fmt-option decode_md=0
--output-fmt cram,version=3.0 --output-fmt-option embed_ref
--output-fmt-option seqs_per_slice=2000 -o foo.cram foo.bam

samtools view -O cram,small -o bar.cram bar.bam
```

 The **--write-index** option enables automatic index creation while writing out BAM, CRAM or bgzf SAM files. Note to get compressed SAM as the output format you need to manually request a compression level, otherwise all SAM files are uncompressed. By default SAM and BAM will use CSI indices while CRAM will use CRAI indices. If you need to create BAI indices note that it is possible to specify the name of the index being written to, and hence the format, by using the **filename##idx##indexname** notation.

For example: to convert a BAM to a compressed SAM with CSI indexing:

```
samtools view -h -O sam,level=6 --write-index in.bam -o out.sam.gz
```

To convert a SAM to a compressed BAM using BAI indexing:

```
samtools view --write-index in.sam -o out.bam##idx##out.bam.bai
```

The **--verbosity** *INT* option sets the verbosity level for samtools and HTSLib. The default is 3 (HTS\_LOG\_WARNING); 2 reduces warning messages and 0 or 1 also reduces some error messages, while values greater than 3 produce increasing numbers of additional warnings and logging messages.

## FILTER EXPRESSIONS

Filter expressions are used as an on-the-fly checking of incoming SAM, BAM or CRAM records, discarding records that do not match the specified expression.

The language used is primarily C style, but with a few differences in the precedence rules for bit operators and the inclusion of regular expression matching.

The operator precedence, from strongest binding to weakest, is:

|               |                       |                                         |
|---------------|-----------------------|-----------------------------------------|
| Grouping      | (, )                  | E.g. "(1+2)*3"                          |
| Values:       | <b>literals, vars</b> | Numbers, strings and variables          |
| Unary ops:    | +, -, !, ~            | E.g. -10 +10, !10 (not), ~5 (bit not)   |
| Math ops:     | *, /, %               | Multiply, division and (integer) modulo |
| Math ops:     | +, -                  | Addition / subtraction                  |
| Bit-wise:     | &                     | Integer AND                             |
| Bit-wise      | ^                     | Integer XOR                             |
| Bit-wise      |                       | Integer OR                              |
| Conditionals: | >, >=, <, <=          |                                         |
| Equality:     | ==, !=, =~, !~        | =~ and !~ match regular expressions     |
| Boolean:      | &&,                   | Logical AND / OR                        |

Expressions are computed using floating point mathematics, so "10 / 4" evaluates to 2.5 rather than 2. They may be written as integers in decimal or "0x" plus hexadecimal, and floating point with or without exponents. However operations that require integers first do an implicit type conversion, so "7.9 % 5" is 2 and "7.9 & 4.1" is equivalent to "7 & 4", which is 4. Strings are always specified using double quotes. To get a double quote in a string, use backslash. Similarly a double backslash is used to get a literal backslash. For example **ab"c\d** is the string **ab"c\d**.

Comparison operators are evaluated as a match being 1 and a mismatch being 0, thus "(2 > 1) + (3 < 5)" evaluates as 2. All comparisons involving undefined (null) values are deemed to be false.

The variables are where the file format specifics are accessed from the expression. The variables correspond to SAM fields, for example to find paired alignments with high mapping quality and a very large insert size, we may use the expression "**mapq >= 30 && (tlen >= 100000 || tlen <= -100000)**". Valid variable names and their data types are:

|               |     |                                  |
|---------------|-----|----------------------------------|
| <b>endpos</b> | int | Alignment end position (1-based) |
|---------------|-----|----------------------------------|

|                                                                                             |              |                                       |
|---------------------------------------------------------------------------------------------|--------------|---------------------------------------|
|  <b>flag</b> | int          | Combined FLAG field                   |
| <b>flag.paired</b>                                                                          | int          | Single bit, 0 or 1                    |
| <b>flag.proper_pair</b>                                                                     | int          | Single bit, 0 or 2                    |
| <b>flag.unmap</b>                                                                           | int          | Single bit, 0 or 4                    |
| <b>flag.munmap</b>                                                                          | int          | Single bit, 0 or 8                    |
| <b>flag.reverse</b>                                                                         | int          | Single bit, 0 or 16                   |
| <b>flag.mreverse</b>                                                                        | int          | Single bit, 0 or 32                   |
| <b>flag.read1</b>                                                                           | int          | Single bit, 0 or 64                   |
| <b>flag.read2</b>                                                                           | int          | Single bit, 0 or 128                  |
| <b>flag.secondary</b>                                                                       | int          | Single bit, 0 or 256                  |
| <b>flag.qcfail</b>                                                                          | int          | Single bit, 0 or 512                  |
| <b>flag.dup</b>                                                                             | int          | Single bit, 0 or 1024                 |
| <b>flag.supplementary</b>                                                                   | int          | Single bit, 0 or 2048                 |
| <b>hclen</b>                                                                                | int          | Number of hard-clipped bases          |
| <b>library</b>                                                                              | string       | Library (LB header via RG)            |
| <b>mapq</b>                                                                                 | int          | Mapping quality                       |
| <b>mpos</b>                                                                                 | int          | Synonym for pnext                     |
| <b>mrefid</b>                                                                               | int          | Mate reference number (0 based)       |
| <b>mrname</b>                                                                               | string       | Synonym for rnext                     |
| <b>ncigar</b>                                                                               | int          | Number of cigar operations            |
| <b>pnext</b>                                                                                | int          | Mate's alignment position (1-based)   |
| <b>pos</b>                                                                                  | int          | Alignment position (1-based)          |
| <b>qlen</b>                                                                                 | int          | Alignment length: no. query bases     |
| <b>qname</b>                                                                                | string       | Query name                            |
| <b>qual</b>                                                                                 | string       | Quality values (raw, 0 based)         |
| <b>refid</b>                                                                                | int          | Integer reference number (0 based)    |
| <b>rlen</b>                                                                                 | int          | Alignment length: no. reference bases |
| <b>rname</b>                                                                                | string       | Reference name                        |
| <b>rnext</b>                                                                                | string       | Mate's reference name                 |
| <b>sclen</b>                                                                                | int          | Number of soft-clipped bases          |
| <b>seq</b>                                                                                  | string       | Sequence                              |
| <b>tlen</b>                                                                                 | int          | Template length (insert size)         |
| <b>[XX]</b>                                                                                 | int / string | XX tag value                          |

Flags are returned either as the whole flag value or by checking for a single bit. Hence the filter expression **flag.dup** is equivalent to **flag & 1024**.

"qlen" and "rlen" are measured using the CIGAR string to count the number of query (sequence) and reference bases consumed. Note "qlen" may not exactly match the length of the "seq" field if the sequence is **"\*"**.

"sclen" and "hclen" are the number of soft and hard-clipped bases respectively. The formula "qlen-sclen" gives the number of sequence bases used in the alignment, distinguishing between global alignment and local alignment length.

"endpos" is the (1-based inclusive) position of the rightmost mapped base of the read, as measured using the CIGAR string, and for mapped reads is equivalent to "pos+rlen-1". For unmapped reads, it is the same as "pos".

Reference names may be matched either by their string forms ("rname" and "mrname") or as the Nth **@SQ** line (counting from zero) as stored in BAM using "tid" and "mtid" respectively.

Auxiliary tags are described in square brackets and these expand to either integer or string as defined by the tag itself (**XX:Z:string** or **XX:i:int**). For example **[NM]>=10** can be used to look for alignments with many mismatches and **[RG]=~"grp[ABC]-"** will match the read-group string.

If no comparison is used with an auxiliary tag it is taken simply to be a test for the existence of that tag. So **[NM]** will return any record containing an NM tag, even if that tag is zero (**NM:i:0**). In htlib <= 1.15 negating this with **![NM]** gave misleading results as it was true if the tag did not exist or did exist but was zero. Now this is strictly does-not-exist. An explicit **exists([NM])** and **!exists([NM])** function has also been added to make this intention clear.

Similarly in htlib <= 1.15 using **[NM]!=0** was true both when the tag existed and was not zero as well as when the tag did not exist. From 1.16 onwards all comparison operators are only true for tags that exist, so **[NM]!=0** works as expected.

Some simple functions are available to operate on strings. These treat the strings as arrays of bytes, permitting their length, minimum, maximum and average values to be computed. These are useful for processing Quality Scores.

**length(x)** Length of the string (excluding nul char)  
**min(x)** Minimum byte value in the string  
**max(x)** Maximum byte value in the string  
**avg(x)** Average byte value in the string

Note that "avg" is a floating point value and it may be NAN for empty strings. This means that "avg(qual)" does not produce an error for records that have both seq and qual of "\*". NAN values will fail any conditional checks, so e.g. "avg(qual) > 20" works and will not report these records. NAN also fails all equality, < and > comparisons, and returns zero when given as an argument to the **exists** function. It can be negated with **!x** in which case it becomes true.

Functions that operate on both strings and numerics:

**exists(x)** True if the value exists (or is explicitly true).  
**default(x,d)** Value **x** if it exists or **d** if not.

Functions that apply only to numeric values:

**sqrt(x)** Square root of **x**  
**log(x)** Natural logarithm of **x**  
**pow(x, y)** Power function, **x** to the power of **y**  
**exp(x)** Base-e exponential, equivalent to **pow(e,x)**

## ENVIRONMENT VARIABLES

**HTS\_PATH** A colon-separated list of directories in which to search for HTSlib plugins. If \$HTS\_PATH starts or ends with a colon or contains a double colon (::), the built-in list of directories is searched at that point in the search.

If no HTS\_PATH variable is defined, the built-in list of directories specified when HTSlib was built is used, which typically includes **/usr/local/libexec/htslib** and similar directories.

**REF\_PATH** A colon separated (semi-colon on Windows) list of locations in which to look for sequences identified by their MD5sums. This can be either a list of directories or URLs. Note that if a URL is included then the colon in http:// and ftp:// and the optional port number will be treated as part of the URL and not a PATH field separator. Alternatively a double colon may be used to indicate a single colon character. If REF\_PATH includes **%nums** then it is replaced with the next **num** elements of the md5sum. An implicit **/%s** is also added to each path element if any md5sum digits are unused. For example "REF\_PATH=/some/dir/%4s/%s" or "REF\_PATH=/some/dir/%4s" will search a directory structure with the first 4 characters of the md5sum as a subdirectory and the remaining 28 as the filename within that directory.





Version 1.21 and earlier defaulted to using the EBI's CRAM reference server if no REF\_PATH was specified. This default has been removed to reduce load on the EBI's service. It is recommended that a site-wide proxy is set up to allow better sharing of downloaded references, for example the *ref-cache* server provided with HTSlib. The original behaviour can be restored by including <http://www.ebi.ac.uk/ena/cram/md5/%s> in your REF\_PATH. If that is done, it is strongly encouraged you also specify a local REF\_CACHE directory.

See <[https://www.htslib.org/doc/reference\\_seqs.html](https://www.htslib.org/doc/reference_seqs.html) ([https://www.htslib.org/doc/reference\\_seqs.html](https://www.htslib.org/doc/reference_seqs.html))> and **REFERENCE SEQUENCES** below for more information.

**REF\_CACHE** This can be defined to a single location housing a local cache of references. When REF\_CACHE is set any non-local reference will create a file in the local REF\_CACHE named after the sequence md5sum. This cache will be searched prior to REF\_PATH. If you wish to search REF\_CACHE but not to further populate it, add the directory to the start of REF\_PATH instead.

As per REF\_PATH, the percent notation (e.g. "dir/%2s/%2s/%s") may be used to avoid too many files within a single directory.

To pre-populate the REF\_CACHE a script **misc/seq\_cache\_populate.pl** is provided in the Samtools distribution. This takes a fasta file or a directory of fasta files and generates the MD5sum named files.

For example if you use **seq\_cache\_populate -subdirs 2 -root /local/ref\_cache** to create 2 nested subdirectories (the default), each consuming 2 characters of the MD5sum, then REF\_CACHE must be set to **/local/ref\_cache/%2s/%2s/%s**.

## REFERENCE SEQUENCES

The CRAM format requires use of a reference sequence for both reading and writing.

When reading a CRAM the **@SQ** headers are interrogated to identify the reference sequence MD5sum (**M5:** tag) and the local reference sequence filename (**UR:** tag). Note that non-local URIs in the UR tag are not used, but *file://* is supported. This is a change in behaviour, but not documentation, to htslib 1.21.

To create a CRAM the **@SQ** headers will also be read to identify the reference sequences, but M5: and UR: tags may not be present. In this case the **-T** and **-t** options of samtools view may be used to specify the fasta or fasta.fai filenames respectively (provided the .fasta.fai file is also backed up by a .fasta file).

The search order to obtain a reference is:

Use any local file specified by the command line options (eg -T).

Look for MD5 via REF\_CACHE environment variable.

Look for MD5 in each element of the REF\_PATH environment variable.

Look for a local file listed in the UR: header tag.

## EXAMPLES

- Import SAM to BAM when **@SQ** lines are present in the header:

```
samtools view -b a1n.sam > a1n.bam
```

If **@SQ** lines are absent:





```
samtools faidx ref.fa
samtools view -bt ref.fa.fai aln.sam > aln.bam
```

where *ref.fa.fai* is generated automatically by the **faidx** command.

- Convert a BAM file to a CRAM file using a local reference sequence.

```
samtools view -C -T ref.fa aln.bam > aln.cram
```

## AUTHOR

Heng Li from the Sanger Institute wrote the original C version of samtools. Bob Handsaker from the Broad Institute implemented the BGZF library. Petr Danecek and Heng Li wrote the VCF/BCF implementation. James Bonfield from the Sanger Institute developed the CRAM implementation. Other large code contributions have been made by John Marshall, Rob Davies, Martin Pollard, Andrew Whitwham, Valeriu Ohan, Vasudeva Sarma (all while primarily at the Sanger Institute), with numerous other smaller but valuable contributions. See the per-command manual pages for further authorship.

## SEE ALSO

*samtools-addreplacerg* ([samtools-addreplacerg.html](#)) (1), *samtools-ampliconclip* ([samtools-ampliconclip.html](#)) (1), *samtools-ampliconstats* ([samtools-ampliconstats.html](#)) (1), *samtools-bedcov* ([samtools-bedcov.html](#)) (1), *samtools-calmd* ([samtools-calmd.html](#)) (1), *samtools-cat* ([samtools-cat.html](#)) (1), *samtools-checksum* ([samtools-checksum.html](#)) (1), *samtools-collate* ([samtools-collate.html](#)) (1), *samtools-consensus* ([samtools-consensus.html](#)) (1), *samtools-coverage* ([samtools-coverage.html](#)) (1), *samtools-cram-size* ([samtools-cram-size.html](#)) (1), *samtools-depad* ([samtools-depad.html](#)) (1), *samtools-depth* ([samtools-depth.html](#)) (1), *samtools-dict* ([samtools-dict.html](#)) (1), *samtools-faidx* ([samtools-faidx.html](#)) (1), *samtools-fasta* ([samtools-fasta.html](#)) (1), *samtools-fastq* ([samtools-fastq.html](#)) (1), *samtools-fixmate* ([samtools-fixmate.html](#)) (1), *samtools-flags* ([samtools-flags.html](#)) (1), *samtools-flagstat* ([samtools-flagstat.html](#)) (1), *samtools-fqidx* ([samtools-fqidx.html](#)) (1), *samtools-head* ([samtools-head.html](#)) (1), *samtools-idxstats* ([samtools-idxstats.html](#)) (1), *samtools-import* ([samtools-import.html](#)) (1), *samtools-index* ([samtools-index.html](#)) (1), *samtools-markdup* ([samtools-markdup.html](#)) (1), *samtools-merge* ([samtools-merge.html](#)) (1), *samtools-mpileup* ([samtools-mpileup.html](#)) (1), *samtools-phase* ([samtools-phase.html](#)) (1), *samtools-quickcheck* ([samtools-quickcheck.html](#)) (1), *samtools-reference* ([samtools-reference.html](#)) (1), *samtools-reheader* ([samtools-reheader.html](#)) (1), *samtools-reset* ([samtools-reset.html](#)) (1), *samtools-rmdup* ([samtools-rmdup.html](#)) (1), *samtools-sort* ([samtools-sort.html](#)) (1), *samtools-split* ([samtools-split.html](#)) (1), *samtools-stats* ([samtools-stats.html](#)) (1), *samtools-targetcut* ([samtools-targetcut.html](#)) (1), *samtools-tview* ([samtools-tview.html](#)) (1), *samtools-view* ([samtools-view.html](#)) (1), *bcftools* ([bcftools.html](#)) (1), *sam* ([sam.html](#)) (5), *tabix* ([tabix.html](#)) (1) *ref-cache*(1)

Samtools website: <<http://www.htslib.org/>> (<http://www.htslib.org/>)>

File format specification of SAM/BAM,CRAM,VCF/BCF: <<http://samtools.github.io/hts-specs>> (<http://samtools.github.io/hts-specs>)>

Samtools latest source: <<https://github.com/samtools/samtools>> (<https://github.com/samtools/samtools>)>

HTSlib latest source: <<https://github.com/samtools/htslib>> (<https://github.com/samtools/htslib>)>

Bcftools website: <<http://samtools.github.io/bcftools>> (<http://samtools.github.io/bcftools>)>